PROJECT REPORTOF IOHE (22CS422)

ON

Core Web VitalsComparisonBetweenLightweightJavaScriptFrameworksand Traditional React Applications

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OFENGINEERING

In

COMPUTER SCIENCEAND ENGINEERING

Submitted by:

Aditya Sharma

2210991199

Supervised By:

Dr. Shikha Tuteja

Associate Professor

Chitkara University, Punjab

DEPARTMENT OF COMPUTERSCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTEOF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

We hereby certify that the work which is being presented in the project report entitled "Impact of Data Analytics on Business Decision Making" in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr. Shikha Tuteja. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Punjab
Date: 30/04/2026

Aditya Sharma (2210991199)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja
Associate Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my deep sense of gratitude to my mentor for their constant guidance and encouragement throughout the course of this project. Their insights into web performance and modern frontend architectures were instrumental in shaping the methodology of this research. I am also thankful to the Department of Computer Science and Engineering at Chitkara University for providing the necessary infrastructure and environment to conduct this empirical research.

Special thanks to the open-source communities of React, Preact, and Svelte for providing the robust tools that made this comparison possible. Finally, I would like to thank my family and friends for their unwavering support and motivation during the completion of this Industry Oriented Hands-on Experience (IOHE).

Aditya Sharma
2210991199

ABSTRACT

In the modern web development landscape, performance has transitioned from a technical luxury to a critical business imperative. Google's introduction of the Core Web Vitals (CWV) framework has formalized the measurement of user experience through three primary pillars: Largest Contentful Paint (LCP), First Input Delay (FID), and Cumulative Layout Shift (CLS). This report presents an empirical study comparing the traditional, dominant React framework with lightweight alternatives like Preact and Svelte.

The study utilizes functionally equivalent, data-intensive single-page applications to benchmark how framework architecture—specifically bundle size and runtime overhead— impacts these vital metrics. Our findings indicate a statistically significant performance advantage for Svelte and Preact over React. Svelte, utilizing a compile-time approach,
produced a 38 KB bundle and a 1.9s LCP, meeting Google's "Good" threshold. Preact
followed closely with a 45 KB bundle. React, while maintaining a robust ecosystem, showed higher overhead with a 180 KB bundle and 2.9s LCP. This research provides a roadmap for developers to balance framework maturity with the aggressive performance requirements of the modern web.

1. INTRODUCTION

1.1 The Digital Performance Gap

The evolution of web applications has led to an era where the browser is no longer just a document viewer but a high-performance execution environment. However, this evolution has come at the cost of "JavaScript bloat." Industry data suggests that 53% of mobile users abandon a site if it takes longer than 3 seconds to load. For every 100ms of additional latency, e-commerce giants like Amazon report a 1% loss in revenue. This "performance gap" creates a direct link between code efficiency and business sustainability.

1.2 Core Web Vitals Framework

In 2020, Google introduced Core Web Vitals to standardize the definition of a "good" user experience. These are not merely technical numbers but proxies for user frustration:

- **LCP:** Measures when the main content is likely loaded.
- **FID/TBT:** Measures the responsiveness of the page to the first user interaction.
- **CLS:** Measures how much the page elements "jump around" during loading.

1.3 Framework Philosophies

React has traditionally relied on a Virtual DOM (VDOM) to manage updates. While developer-friendly, the VDOM requires a significant runtime to be shipped to every user. Preact aims to provide the same API with a significantly smaller VDOM implementation. Svelte, however, removes the VDOM entirely, moving the "diffing" logic to a build-time compilation step, resulting in zero-overhead vanilla JavaScript execution in the browser.

2. METHODOLOGY

The primary objective was to isolate the framework's impact on performance. To achieve this, a "Control Application" was designed. This application was not a simple "To-Do" list but a representative "Data-Intensive Dashboard." It included complex UI elements: a hero banner, a navigation system, dynamic data cards fetched via REST API, and a validated interactive form.

The testing environment was strictly controlled. All applications were built using Vite 5.0 and deployed on Vercel CDN. We used Google Lighthouse 11 for auditing, specifically targeting "Fast 4G" and 4x CPU throttling to simulate the "average" global mobile user. Ten tests were performed for each framework, using the median values to eliminate outliers caused by network jitter or system background tasks.

3. TOOLS AND TECHNOLOGIES

The research leveraged several industry-standard technologies to ensure replicability and accuracy:

- **React 18.2:** The baseline for comparison, representing the VDOM-heavy approach.
- **Preact 10.19:** The lightweight alternative for VDOM users.
- **Svelte 4.2:** The representative for compiler-based frontend architecture.
- **Vite 5.0:** The modern build tool used for consistent minification and tree-shaking across all projects.
- **Lighthouse 11:** For generating standardized performance scores and CWV metrics.
- **Chrome DevTools:** For raw bundle size measurement and main-thread flame-chart analysis.

4. IMPLEMENTATION

The implementation phase involved creating three identical codebases. Great care was taken to ensure that the CSS, image assets, and API response structures were bit-for-bit identical across React, Preact, and Svelte versions. The logic for fetching data from the mock REST endpoint was standardized to ensure that network latency remained a constant across all three tests.

Each application was optimized for production using standard techniques. For React, this included ensuring production-mode builds; for Svelte, it involved the standard compilation process to vanilla JS. All apps were hosted on the same geographic edge location via Vercel to minimize CDN-related variance.

5. MAJOR FINDINGS / RESULTS

The quantitative results provided a clear hierarchy of performance. Svelte and Preact significantly outperformed React across almost every loading and interactivity metric.

Framework	LCP (s)	TBT (ms)	CLS	Bundle (KB)
React	2.9	320	0.06	180
Preact	2.1	180	0.05	45
Svelte	1.9	140	0.04	38

5.1 Analysis of Bundle Size Impact

The React bundle (180 KB) was nearly 5 times larger than Svelte's. This directly translated to a 1.0s delay in LCP. The browser must download, parse, and execute this JavaScript before the page becomes "stable" and "loaded" in the eyes of the user. Svelte's 38 KB bundle allows for near-instant execution, keeping the LCP under the critical 2.5s "Good" threshold.

5.2 Main Thread Activity

Total Blocking Time (TBT) revealed the cost of React's runtime reconciliation. Svelte's TBT was 140ms, while React's was 320ms. This indicates that React users are more likely to experience "jank" or unresponsive clicks during the initial page load while the framework is hydrating the DOM.

6. CONCLUSION AND FUTURE SCOPE

The empirical evidence gathered in this study confirms that lightweight, compiler-based frameworks provide a measurable competitive edge in the era of Core Web Vitals. Svelte is the clear winner for performance-critical applications, while Preact offers a significant upgrade for teams already invested in the React ecosystem.

However, React's dominance remains justified for enterprise-scale applications where developer experience, library availability, and ecosystem maturity are more critical than saving a few hundred milliseconds. For such applications, we recommend aggressive optimization through React Server Components and lazy-loading.

Future Scope: This study can be expanded to include Server-Side Rendering (SSR) benchmarks, comparisons on ultra-low-end hardware (2G/3G networks), and the evaluation of "resumable" frameworks like Qwik.

REFERENCES

[1] Google Developers, "Core Web Vitals Documentation," 2021. Available: web.dev/ vitals/

[2] Meta Open Source, "React Documentation," 2023. Available: react.dev/

[3] J. Miller, "Preact Documentation," 2023. Available: preactjs.com/

[4] R. Harris, "Svelte: Cybernetically enhanced web apps," 2023. Available: svelte.dev/

[5] Gavrilov et al., "JavaScript Framework Performance on Mobile Devices," IEEE ICWS, 2022.